

Deep Computer Vision

MIT 6.S191: Introduction to Deep Learning — Lecture 3

Detailed study notes synthesized from the lecture slides and accompanying written notes

Contents

1 What Computer Vision Is Really Trying to Do	1
2 How a Computer Represents an Image	1
3 From Hand-Engineered Features to Learned Features	2
3.1 Hierarchical representation learning	2
4 Why Fully Connected Networks Are Awkward for Images	2
5 Using Spatial Structure: Receptive Fields	3
6 Filters, Kernels, and Feature Detectors	3
6.1 Are filters manually chosen?	4
7 The Convolution Operation	4
8 Feature Maps	5
8.1 Multiple filters in one layer	6
9 Stride, Overlap, and Output Size	6
10 The Basic CNN Architecture	7
10.1 1. Convolution: local weighted computation	7
10.2 2. Non-linearity: ReLU	8
10.3 3. Pooling: spatial downsampling	8
11 Representation Learning Across Depth	9
12 Training the Filters	10
13 CNNs for Image Classification	10
14 One Feature Extractor, Many Vision Tasks	11
14.1 Classification in medicine	11
14.2 Object detection: classify and localize	11
14.2.1 Why a naive sliding/random-box strategy is expensive	12

14.2.2 Region-based CNN idea	12
15 Semantic Segmentation: Prediction at Every Pixel	13
16 End-to-End Autonomous Navigation from Vision	14
17 Compact Mathematical Summary	14
18 Common Points of Confusion	15
19 Lecture 3 Takeaways	15

What Computer Vision Is Really Trying to Do

Computer vision is not only about assigning a name to an object. A rich visual system should reason about **what is present**, **where it is**, **how objects relate to one another**, and potentially **what may happen next**. In a street scene, for example, two visually similar cars can have very different roles: one may be parked while another is temporarily stopped because pedestrians are crossing. The pixels alone do not explicitly state these causal or behavioral relationships; the learning system must extract useful structure from them.

This is why deep computer vision has impact across robotics, mobile computing, accessibility, medicine and biology, and autonomous driving. The common challenge is to transform raw pixel arrays into progressively more useful representations.

Central idea of the lecture: rather than manually specifying every visual feature, deep neural networks learn a *hierarchy of representations* directly from data. Early layers can respond to simple local patterns; deeper layers compose them into increasingly abstract features.

How a Computer Represents an Image

A computer receives an image as numbers. A grayscale image can be represented as a matrix

$$X \in \mathbb{R}^{H \times W},$$

where each entry is a pixel intensity. Depending on preprocessing, intensities may be represented as integers such as 0–255 or normalized values such as $[0, 1]$.

An RGB image adds a channel dimension:

$$X \in \mathbb{R}^{H \times W \times 3},$$

with one 2-D plane each for red, green, and blue. The model therefore does not directly “see” a face, wheel, or road; it initially sees a structured array of numerical values.

Two common output types are:

- **Classification:** predict a discrete class or a probability distribution over classes.
- **Regression:** predict one or more continuous quantities.

From Hand-Engineered Features to Learned Features

Traditional visual pipelines can be described as: domain knowledge $\rightarrow$ define features $\rightarrow$ detect those features $\rightarrow$ classify. This becomes difficult because the appearance of an object changes under viewpoint, illumination, scale, deformation, occlusion, background clutter, and large intra-class variation.

A feature that works for one carefully controlled image may fail under a different camera angle or lighting condition. The deep-learning alternative is to learn both **which features matter** and **how to combine them** from examples.

Learning Feature Representations

Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

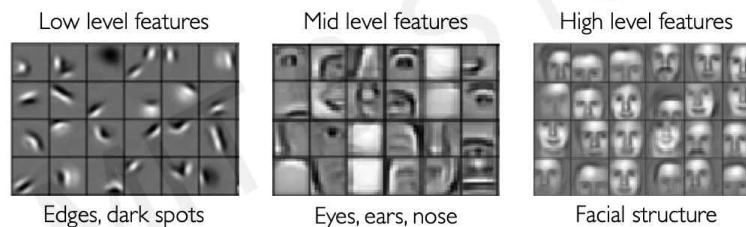


Figure 1: The lecture’s hierarchy-of-features view: low-level edges and dark spots can combine into mid-level facial parts and then high-level facial structure.

Hierarchical representation learning

The hierarchy can be understood conceptually as:

pixels $\rightarrow$ edges/textures $\rightarrow$ parts $\rightarrow$ objects or semantic concepts.

This is not a hard-coded rule saying that every CNN must literally learn “eyes” at one particular layer. It is a useful interpretation of how depth allows local patterns to be composed into larger and more abstract patterns.

Why Fully Connected Networks Are Awkward for Images

A fully connected network expects a vector. A $H \times W$ image can be flattened into a vector of length HW , but flattening discards the explicit 2-D neighborhood structure. Two pixels that were adjacent in the image no longer have a special architectural relationship after flattening.

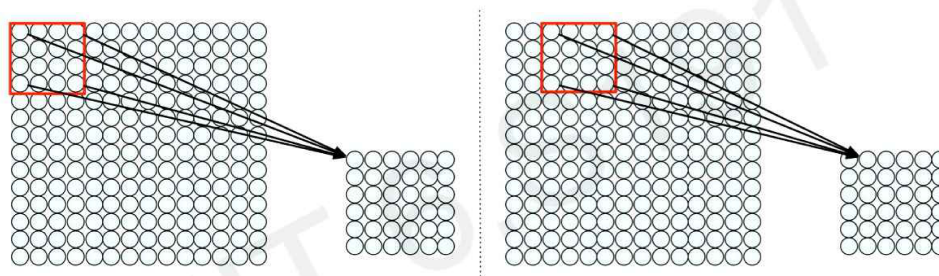
There is also a parameter problem. A 100×100 grayscale image already contains 10,000 input values. Connecting those inputs to only 1,000 hidden units requires roughly 10^7 weights in the first layer alone. Images are large, so dense connectivity quickly becomes inefficient.

CNNs introduce two crucial inductive biases: **local connectivity** (look at a small neighborhood at a time) and **weight sharing** (use the same detector at many spatial locations). These preserve spatial structure while dramatically reducing the number of parameters.

Using Spatial Structure: Receptive Fields

Instead of connecting a hidden neuron to every pixel, connect it only to a local patch. This patch is the neuron's **receptive field**. A neighboring hidden unit looks at a neighboring patch. Sliding this local computation over the image lets the network examine the full image while respecting spatial locality.

Using Spatial Structure



Connect patch in input layer to a single neuron in subsequent layer.

Use a sliding window to define connections.

How can we **weight** the patch to detect particular features?



Figure 2: Local connectivity: a neuron receives information from a spatially local patch rather than from the entire image.

As layers are stacked, the *effective* receptive field grows. A unit in a deeper layer depends on several units in the previous layer, each of which depended on its own local region. This allows deeper units to integrate information over progressively larger parts of the original image.

Filters, Kernels, and Feature Detectors

A convolutional **filter** (or kernel) is a small learnable array of weights. For a single-channel image, a $k \times k$ filter contains k^2 weights. For an RGB input, a filter spans all input channels and has shape $k \times k \times 3$.

The lecture motivates filters using the letter X: two images of X may be quite different pixel-by-pixel, but both can contain similar local diagonal and crossing patterns. Detecting those local patterns gives a representation that is more useful than exact pixel equality.

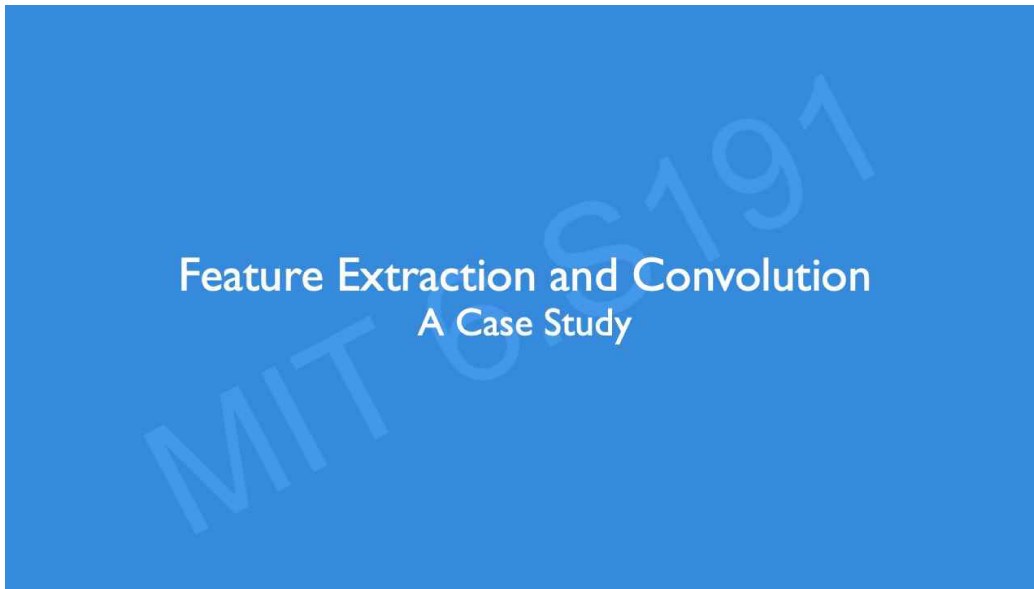


Figure 3: Different pixel arrangements can share the same useful local features. The lecture decomposes an X into diagonal and crossing patterns.

Are filters manually chosen?

The lecture first uses manually specified filters to make the forward operation understandable. In a trained CNN, however, the filter values are **parameters learned from data**, just like weights in other neural networks. They are commonly initialized before training and then updated by gradient-based optimization through backpropagation.

The Convolution Operation

Let an input patch be $X_{i:i+k-1,j:j+k-1}$ and a filter be $K \in \mathbb{R}^{k \times k}$. At one spatial location, the pre-activation is

$$z_{ij} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{uv} X_{i+u,j+v} + b.$$

An activation function ϕ may then be applied:

$$a_{ij} = \phi(z_{ij}).$$

In words: **element-wise multiply the patch and filter, sum the products, add a bias, then apply a non-linearity**. The same filter is reused at other positions.

Features of X

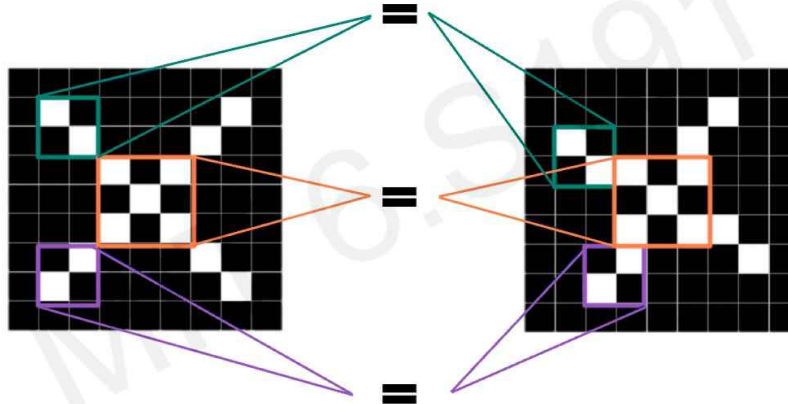


Figure 4: A filter produces a large response when the local image patch aligns strongly with the pattern encoded by the filter.

A strong positive response indicates strong alignment with the learned pattern. The exact interpretation of negative responses depends on the learned filter and subsequent activation; before ReLU, a negative value can represent an opposite or poorly aligned response rather than simply “nothing.”

Feature Maps

Sliding one filter across all valid positions produces a 2-D **feature map**. Each feature-map entry records the response of the same detector at a different spatial location.

The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:

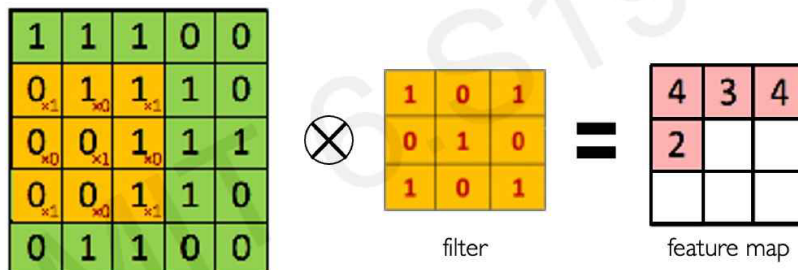


Figure 5: Repeated convolution over the image creates a feature map that indicates where the filter responds strongly.

This directly answers an important conceptual question: **a feature map is not itself the next filter**. It is the *output activation* produced by a filter. If another convolutional layer follows, its new filters operate on the collection of feature maps from the previous layer.

Multiple filters in one layer

A convolutional layer usually learns many filters. If the layer has C_{out} filters, it produces C_{out} output feature maps, which are stacked into an output volume. Thus, one layer can simultaneously learn different detectors.

For an input of shape $H \times W \times C_{\text{in}}$, each filter typically has shape

$$k_h \times k_w \times C_{\text{in}},$$

and one such filter produces one output channel. With C_{out} filters, the kernel tensor has shape

$$k_h \times k_w \times C_{\text{in}} \times C_{\text{out}}.$$

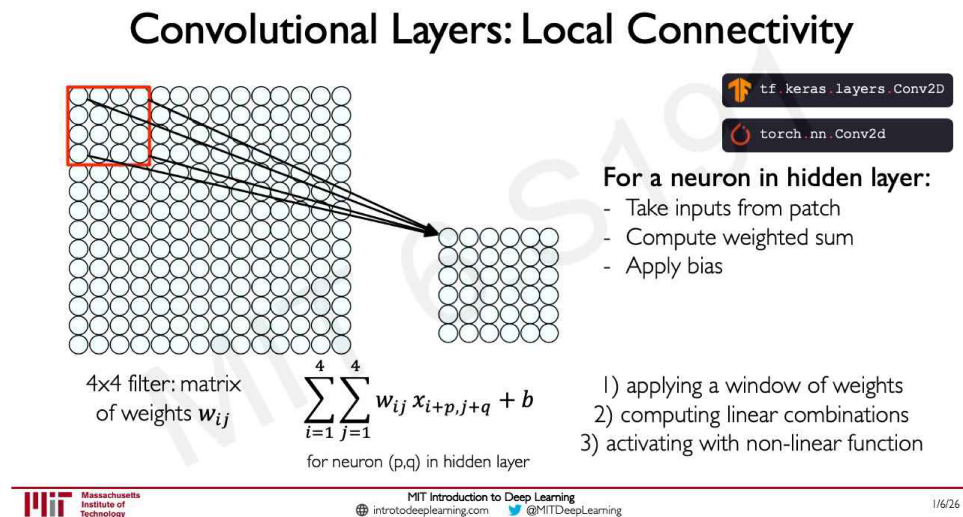


Figure 6: Multiple filters create multiple feature maps; stacking them forms the output volume of the convolutional layer.

Stride, Overlap, and Output Size

The lecture discusses why neighboring filter applications may overlap. The distance by which a filter moves is the **stride** S . A stride of 1 moves one pixel at a time and therefore creates substantial overlap. Larger strides reduce overlap and spatial resolution.

For an input size N , filter size F , zero-padding P , and stride S , the spatial output size is

$$N_{\text{out}} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1.$$

Although the lecture emphasizes overlap conceptually rather than dwelling on this formula, it is useful for keeping track of tensor shapes.

Why overlap can help: visual features do not conveniently align with arbitrary filter boundaries. Overlapping receptive fields reduce the chance that an important pattern is split exactly between two non-overlapping patches.

The Basic CNN Architecture

A CNN for image classification can be viewed as two broad stages:

1. **Feature extraction:** repeated convolution, non-linearity, and often pooling/downsampling.
2. **Task head:** use the learned representation to produce class probabilities or another task-specific output.

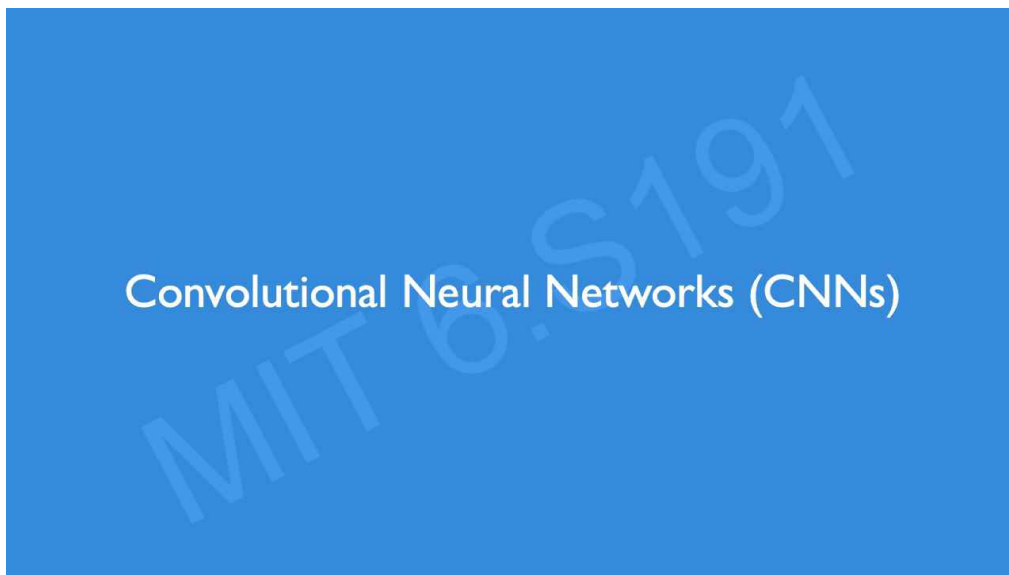


Figure 7: The lecture combines convolution, non-linearity, and pooling into a CNN pipeline for classification.

1. Convolution: local weighted computation

The convolutional layer applies learned filters to local receptive fields. The same weights are reused spatially, making the operation far more parameter-efficient than a fully connected layer over all pixels.

CNNs for Classification

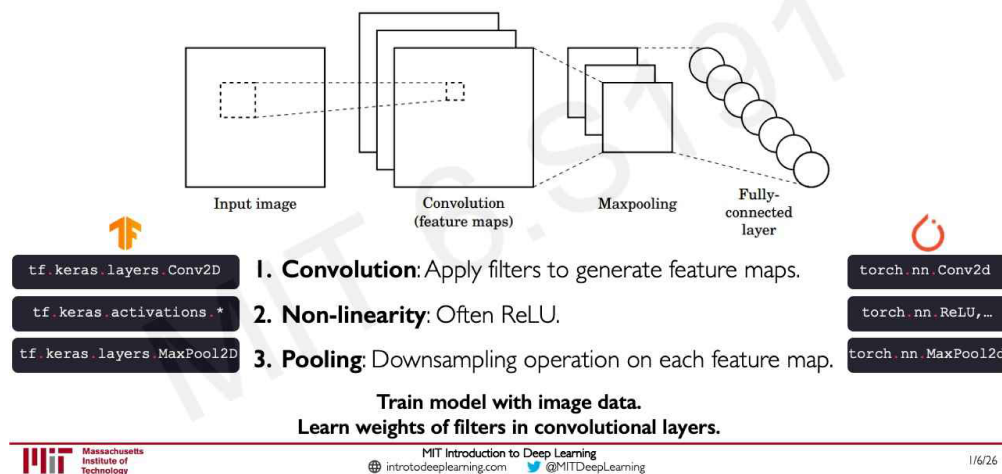


Figure 8: Convolutional layers use local connectivity and shared filter weights.

2. Non-linearity: ReLU

After convolution, CNNs commonly apply a non-linear activation. The lecture uses the Rectified Linear Unit:

$$\text{ReLU}(x) = \max(0, x).$$

ReLU maps negative values to zero and leaves positive values unchanged. Without non-linear activations, stacking linear convolutional operations would still collapse into an overall linear transformation and would severely limit representational power.

CNNs: Spatial Arrangement of Output Volume

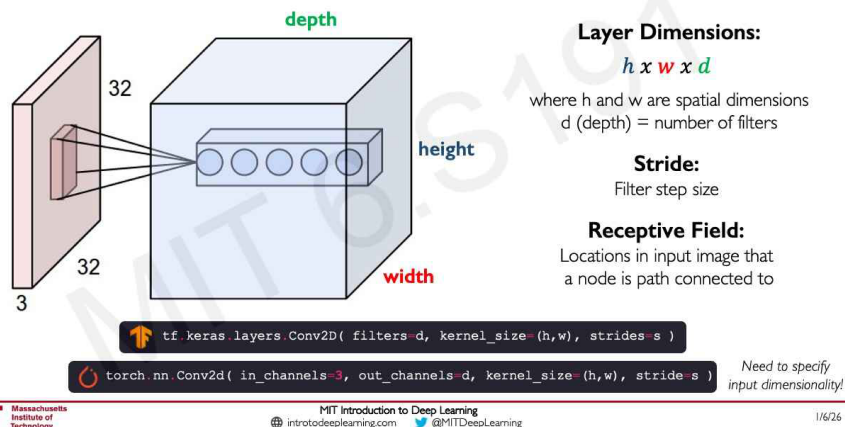


Figure 9: ReLU is applied element-wise to feature maps, suppressing negative activations while preserving positive responses.

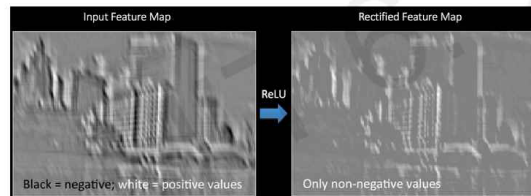
3. Pooling: spatial downsampling

Pooling reduces the spatial dimensions of feature maps. In **max pooling**, each local pooling window is replaced by its maximum value. For example, a 2×2 max-pooling window converts four nearby activations

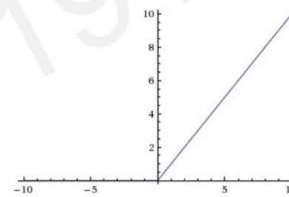
into one value.

Introducing Non-Linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation!**



Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

`tf.keras.layers.ReLU`
`torch.nn.ReLU`

Figure 10: Max pooling downsamples feature maps by retaining the maximum activation in each local window.

Pooling has two important effects emphasized in the lecture: it reduces representation size and helps the network reason across different effective scales as depth increases. Small filters in later, spatially reduced maps correspond to larger regions of the original image.

Representation Learning Across Depth

Early layers operate close to raw pixels and tend to capture relatively simple patterns. Deeper layers receive already-transformed feature maps, allowing them to combine simpler patterns into more abstract structures.

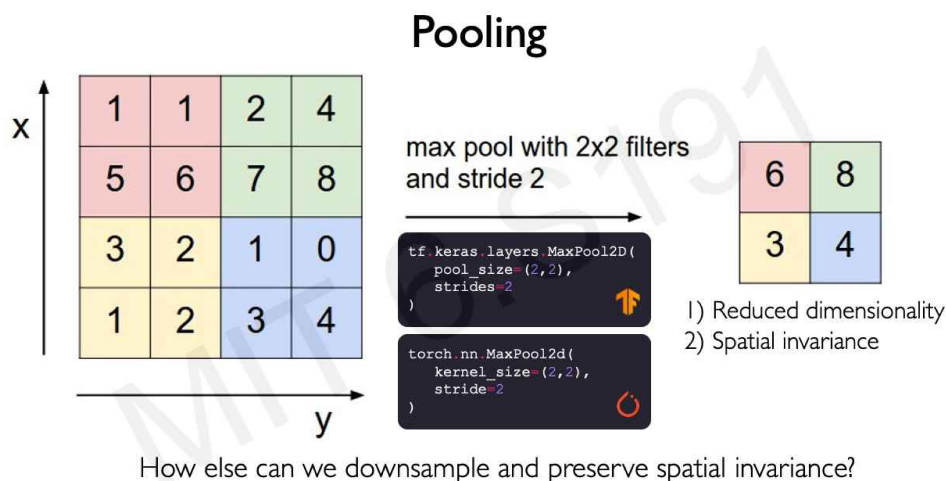


Figure 11: The lecture illustrates the progression from low-level to mid-level to high-level visual features in a deep CNN.

A useful mental model is:

$$\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \dots$$

followed by a task-specific output module. Modern architectures vary substantially, but the principle of learning increasingly useful representations through depth remains central.

Training the Filters

The filter weights are learned end-to-end. For a classification task, the process is:

1. Run an input image through the CNN to obtain class scores/probabilities.
2. Compare the prediction with the target using a loss function.
3. Backpropagate gradients from the loss through the classifier and all convolutional layers.
4. Update filter weights and other parameters with an optimizer.
5. Repeat over many examples.

Thus, a filter is not selected because a human decides “this should be an edge detector.” If an edge-like detector is useful for reducing the training loss, optimization can discover weights that behave that way.

CNNs for Image Classification

For classification, the learned convolutional representation is converted into class scores. A final softmax can turn logits z_c into probabilities:

$$p(y = c \mid x) = \frac{e^{z_c}}{\sum_j e^{z_j}}.$$

The front end learns visual features; the final portion maps those features to the classes required by the problem.

Representation Learning in Deep CNNs

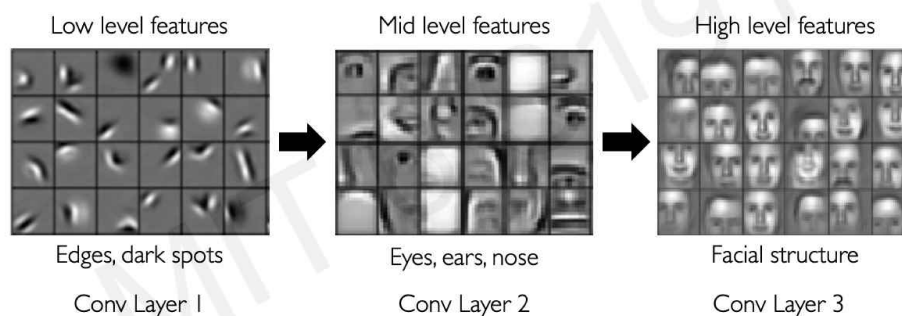


Figure 12: A CNN can use learned feature representations to produce class probabilities.

One Feature Extractor, Many Vision Tasks

A major message near the end of the lecture is that convolutional feature extraction is reusable. The output side changes according to the task.

Putting it all together: CNN in PyTorch

```

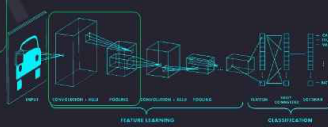
import torch

def generate_model():
    model = nn.Sequential([
        # first and second convolutional layer
        torch.nn.Conv2d(in_channels=3, out_channels=32, filter_size=3),
        torch.nn.ReLU(),
        torch.nn.MaxPool2d(kernel_size=2, stride=2),

        torch.nn.Conv2d(in_channels=32, out_channels=64, filter_size=3),
        torch.nn.ReLU(),
        torch.nn.MaxPool2d(kernel_size=2, stride=2),

        # fully connected classifier
        torch.nn.Flatten(),
        torch.nn.Linear(64*6*6, 1024), # flattened dim after 2 conv layers
        torch.nn.ReLU(),
        torch.nn.Linear(1024, 10), # 10 outputs
    ])
    return model

```



The diagram on the right illustrates the CNN architecture. It starts with an input image (a truck) which is processed by a series of convolutional and pooling layers (labeled 'FEATURE LEARNING'). The resulting feature maps are then flattened and passed through a fully connected layer (labeled 'CLASSIFICATION') to produce the final output (a truck).

Figure 13: A common convolutional feature extractor can support classification, object detection, segmentation, and other prediction tasks.

Classification in medicine

The lecture gives breast-cancer screening as an example of using CNNs to analyze medical images. The task head can produce a classification while the convolutional layers learn visual representations from scans.

Object detection: classify and localize

Classification answers *what* is in an image. Object detection additionally asks *where*. A detector therefore needs to output object categories together with spatial localization, commonly represented by bounding boxes.

A box can be parameterized by location and size, for example (x, y, w, h) , plus class scores. The challenge is that an image may contain an arbitrary number of objects at different positions, scales, and aspect ratios.

Classification: Breast Cancer Screening

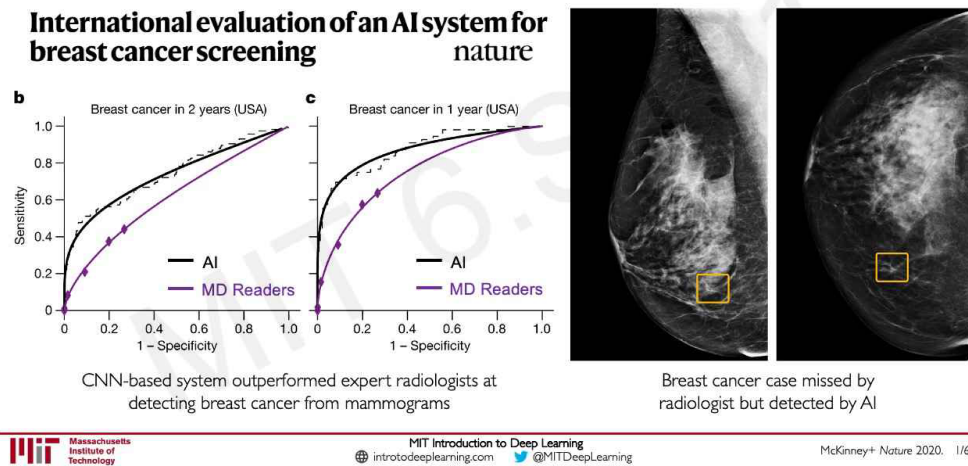


Figure 14: Object detection extends classification by predicting locations/bounding boxes as well as object classes.

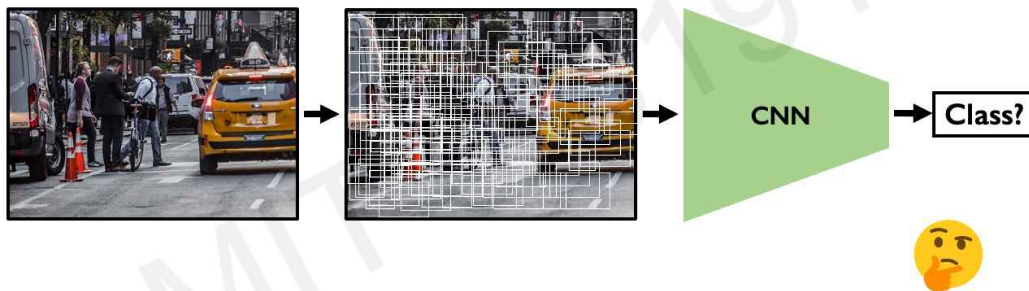
Why a naive sliding/random-box strategy is expensive

Testing huge numbers of candidate crops independently is computationally wasteful. A better strategy is to compute convolutional features for the image and then use those features to reason about candidate regions.

Region-based CNN idea

The lecture introduces the region-based CNN family conceptually: extract convolutional features, generate candidate regions, obtain region-specific features, and classify/localize those regions. Importantly, learning can propagate information through the trainable parts of the pipeline so the model can improve the features used for detection.

Naïve Solution to Object Detection



Problem: Way too many inputs! This results in too many scales, positions, sizes!



Massachusetts
Institute of
Technology

MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning

1/6/26

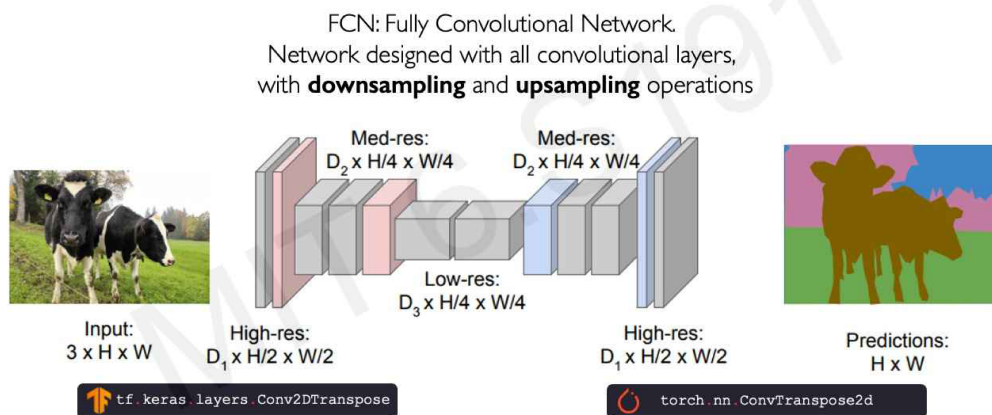
Figure 15: Region-based detection uses convolutional features together with region proposals and per-region prediction.

Semantic Segmentation: Prediction at Every Pixel

Semantic segmentation is an **image-to-image** task. Instead of one label for the whole image or a set of boxes, the network predicts a class for every pixel. If the input is $H \times W \times 3$, the output may be a spatial map whose entries represent classes such as cow, grass, tree, or sky.

The encoder portion progressively extracts and often downsamples features. To return to pixel-level output, the network needs an upsampling/decoder path that restores spatial resolution and produces dense predictions.

Semantic Segmentation: Fully Convolutional Networks



Massachusetts
Institute of
Technology

MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning

Long+ CVPR 2015. 1/6/26

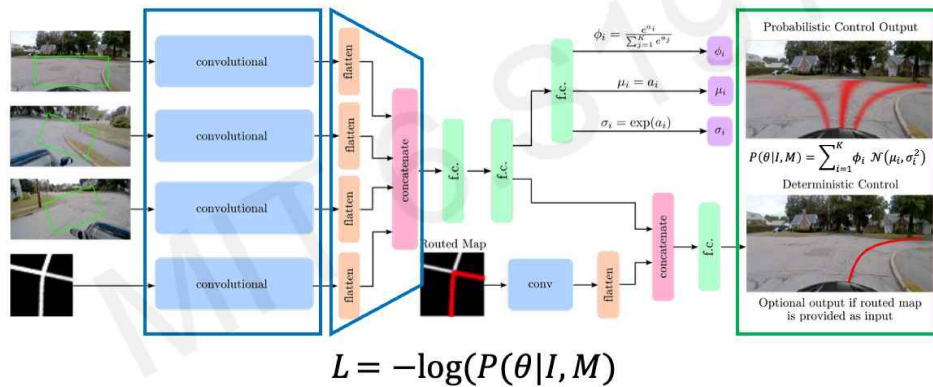
Figure 16: Semantic segmentation predicts a class at each pixel, requiring dense spatial output rather than a single class vector.

End-to-End Autonomous Navigation from Vision

The lecture closes with an autonomous-driving example in which multiple visual inputs and map information can each be processed by convolutional feature extractors. Their learned features can then be combined to predict control-related outputs.

End-to-End Framework for Autonomous Navigation

Entire model is trained end-to-end **without any human labelling or annotations**



MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning

Amini+ ICRA 2019. 1/6/26

Figure 17: Example of an end-to-end vision-to-control architecture for autonomous navigation.

This example reinforces the broader theme: CNNs are not restricted to “image in, class label out.” Convolution provides a general mechanism for extracting structured features, while the downstream head is adapted to the required prediction problem.

Compact Mathematical Summary

For a convolutional layer with input X , filters $K^{(m)}$, biases b_m , and activation ϕ , output channel m is

$$Y_{ij}^{(m)} = \phi \left(\sum_{u,v,c} K_{uvc}^{(m)} X_{i+u,j+v,c} + b_m \right).$$

The important pieces are:

- u, v : local spatial coordinates inside the filter;
- c : input channel index;
- m : output filter / output feature-map index;
- the same $K^{(m)}$ is reused across spatial positions (i, j) ;
- stacking many output maps creates the input volume for the next layer.

Common Points of Confusion

Question	Correct mental model
Is a feature map a filter?	No. A filter is a learned weight tensor. A feature map is the activation produced by applying that filter across an input.
Does one convolutional layer have only one feature map?	Usually no. A layer has many filters and therefore produces many output feature maps/channels.
Does the next layer multiply one feature map by one filter?	A next-layer filter generally spans <i>all</i> channels from the previous layer. It combines information across the previous feature maps while sliding spatially.
Who chooses filter values?	Training learns them. Hand-designed values are useful for demonstrations, but learned CNN filters are model parameters.
Why not flatten the image immediately?	Flattening removes explicit spatial organization and dense layers introduce very large parameter counts.
Why pooling/downsampling?	It reduces spatial size and lets later features correspond to larger regions of the original image.
What changes between classification, detection, and segmentation?	Much of the feature extractor can remain conceptually similar; the task-specific output structure changes.

Lecture 3 Takeaways

1. Images are structured numerical arrays; preserving that spatial structure matters.
2. CNNs replace global dense connectivity with local receptive fields and shared filters.
3. A convolution computes local weighted matches and produces a feature map.
4. Multiple filters create a volume of feature maps, which becomes the input to deeper layers.
5. ReLU supplies non-linearity; pooling/downsampling reduces spatial resolution and increases the effective scale of later features.
6. Depth enables hierarchical representation learning from simple local patterns to more abstract visual structure.
7. The same convolutional feature-learning idea supports classification, object detection, semantic segmentation, and vision-based control.

Deep Learning for Computer Vision: Summary

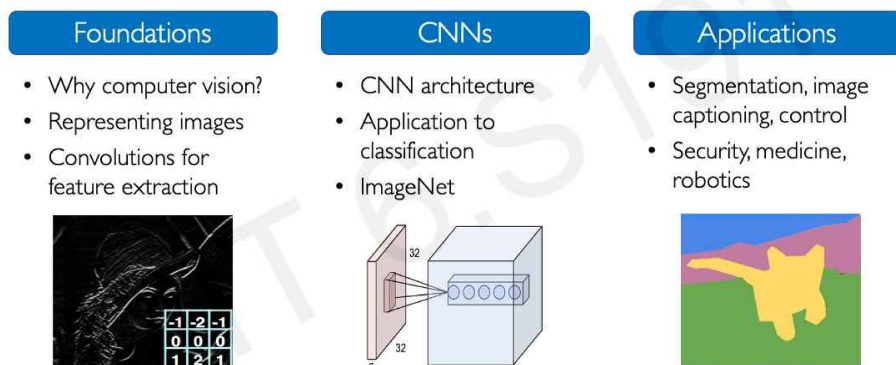


Figure 18: Lecture summary: deep computer vision builds hierarchical learned representations and adapts them to multiple downstream tasks.